# LAB 7 REPORT

# FUZZING FOR INPUT VALIDATION BUGS

**NAME :** SUNIL A. SHINDOLKAR

**SRN:**PES1UG25CS848

# LAB 7: Fuzzing for Input Validation Bugs

## 1. Objective

To explore fuzz testing as a security and quality-assurance technique by identifying and fixing bugs in a Python script using the Hypothesis fuzzing library.

## 2. Tools Used

Python 3.13, pytest, and Hypothesis.

## 3. Brief Theory

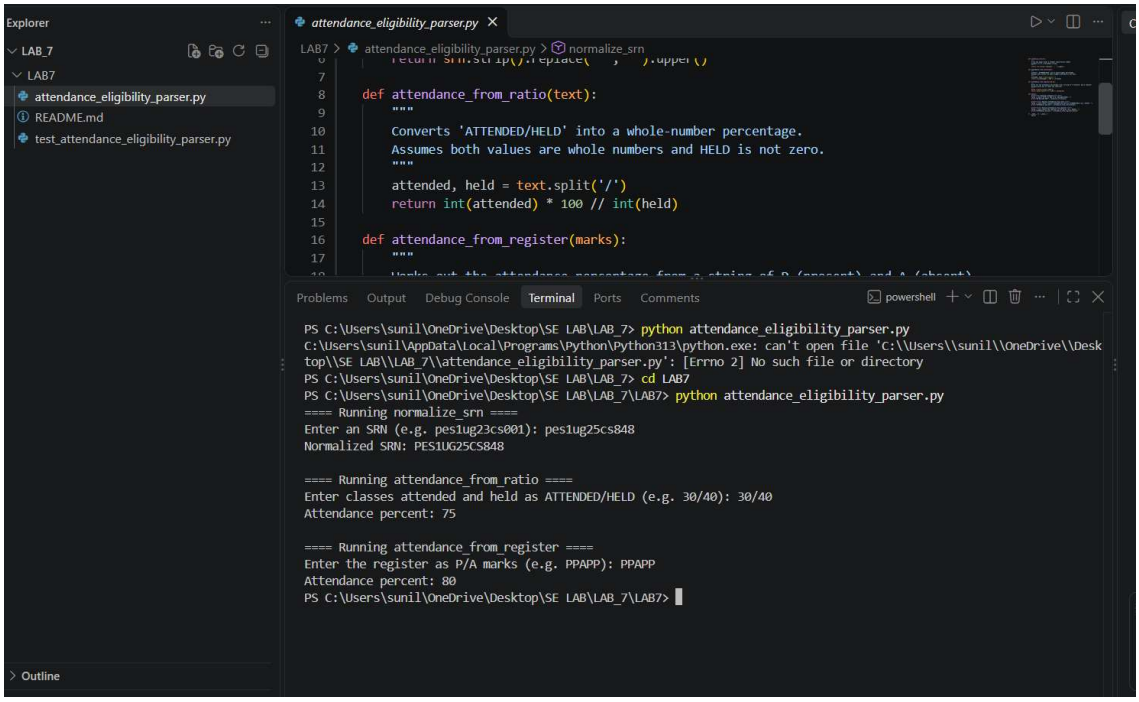
Fuzz testing supplies random, unexpected, or malformed inputs to a program to discover crashes, incorrect behaviour, and edge-case bugs. Hypothesis is a property-based testing library that generates many inputs automatically and helps identify small failing examples.

## 4. Procedure and Execution

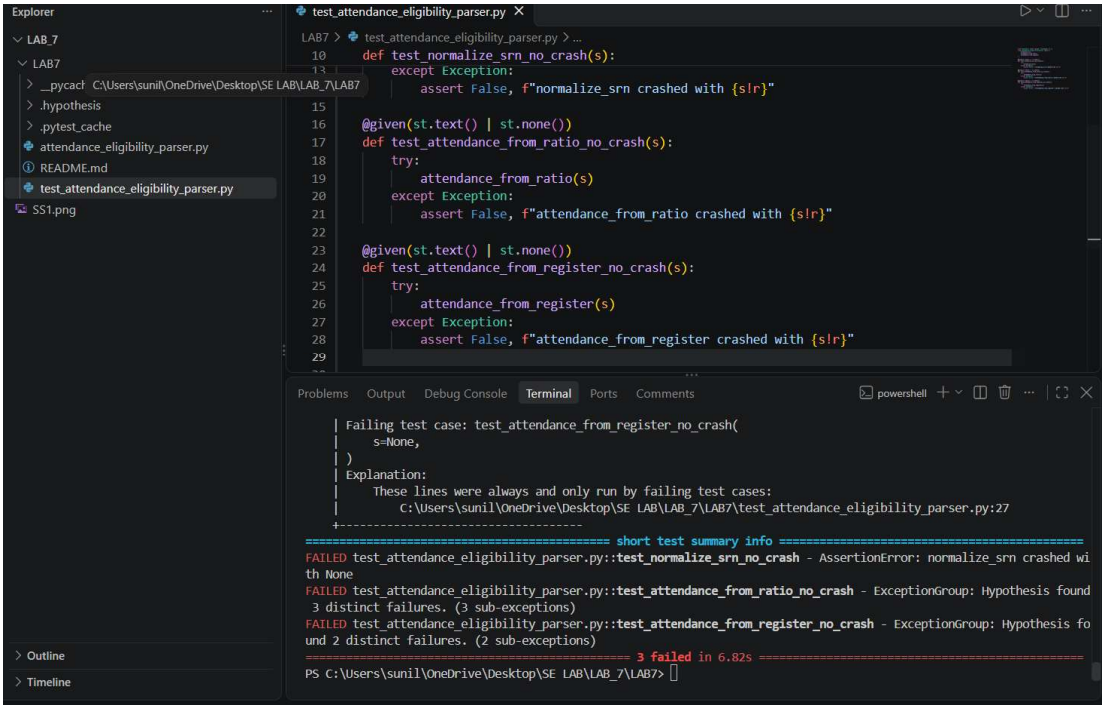
1. Installed the required testing dependencies (Hypothesis and pytest).
2. Executed attendance_eligibility_parser.py and checked the interactive inputs for SRN, attendance ratio, and register marks.
3. Ran the Hypothesis test file. The initial run exposed failures for unexpected inputs such as None.
4. Reviewed and corrected the application functions so they handle unexpected inputs safely.
5. Re-ran the test file and verified that all three tests passed.
6. Executed the corrected application again to confirm normal inputs produce the expected output.

## 5. Screenshots

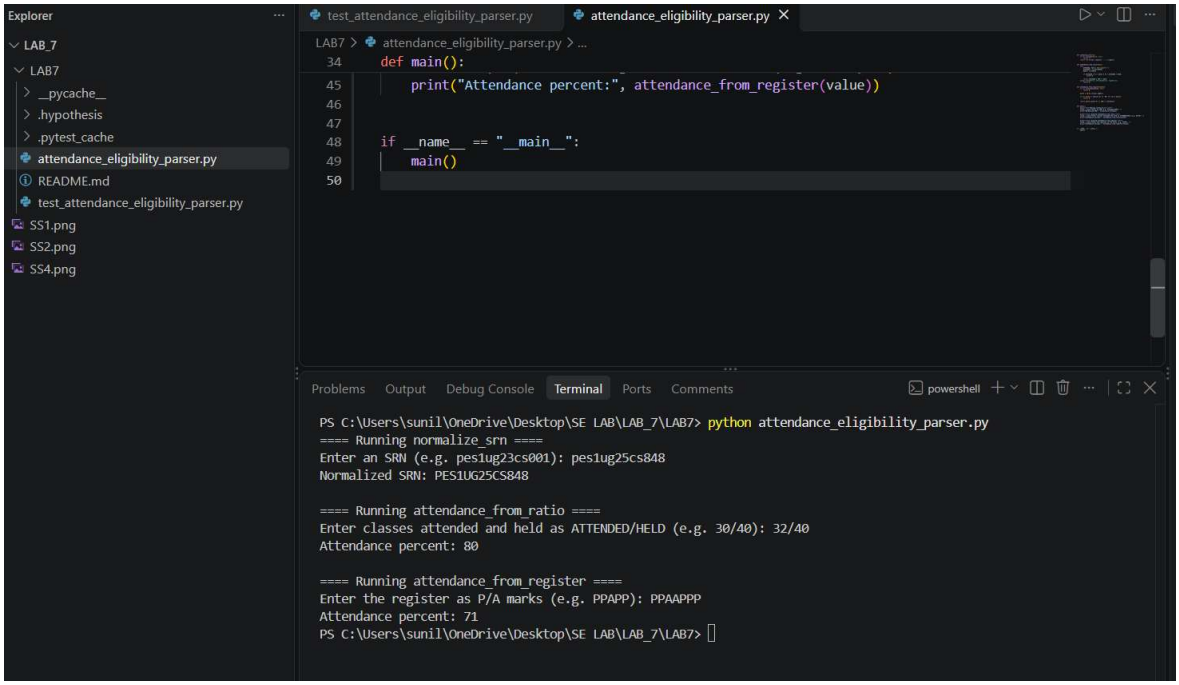
### Screenshot 1 – Initial application execution



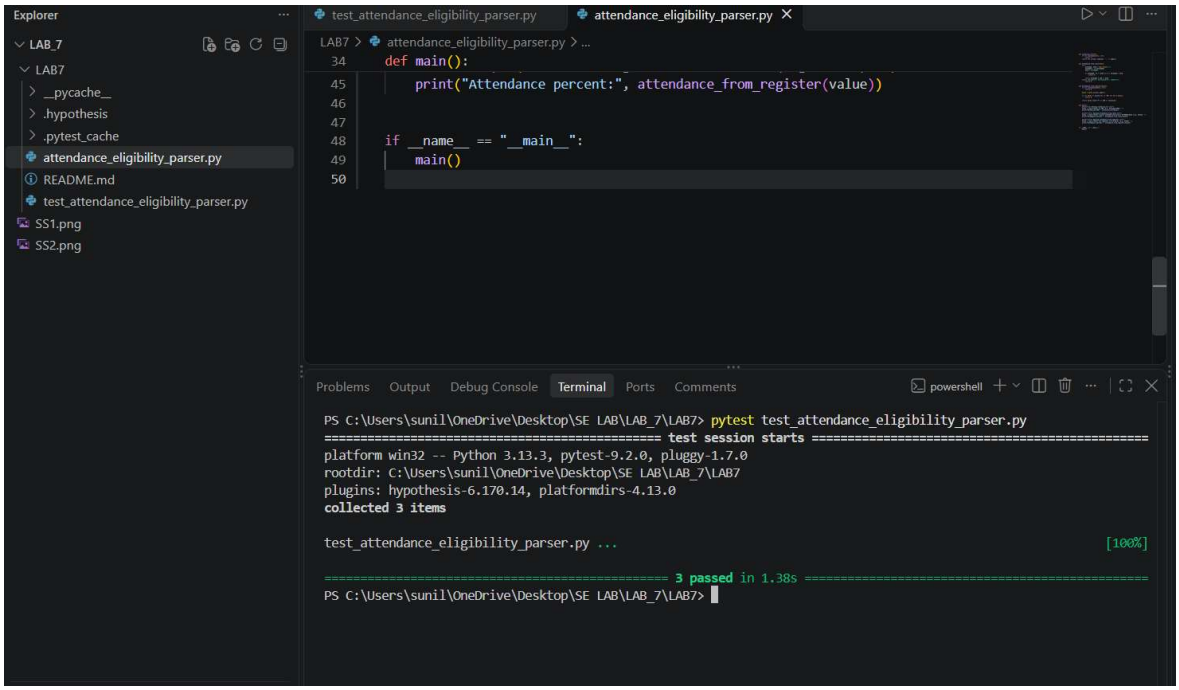

### Screenshot 2 – Initial Hypothesis test failures



## Screenshot 3 – Hypothesis tests passed (3 passed)




## Screenshot 4 – Final application execution



## 6. Result

Fuzz testing was performed using Hypothesis. The initial tests identified input-handling failures, the application was hardened, and the final test execution completed successfully with 3 tests passed. The application was also run again with sample inputs.

## 7. Conclusion

This lab demonstrated how property-based fuzzing can reveal edge cases that may be missed by ordinary manual testing. Defensive input handling and re-running automated tests improved the robustness of the program.